MODULE *ccffix*

FIX layer over *ccfraft.tla*.

Adds a *committed COMMIT − CERTIFICATE history variable*(*adapted from the granular − storage raft.tla* "encoding #2"), *and restates Leader Completeness in the COMMIT − term − keyed certificate form so it is no longer the entry − term approximation that ccfraft s LeaderCompletenessInv is*(*see fix.readme*).

Why a wrapper instead of editing ccfraft.tla s ∼20 actions: *committed is a pure history*(*ghost*)*variable.We add it as a monitor whose update is a function of the observed state change, so the upstream spec is left pristine and there is no risk of mis − framing committed in some action s* UNCHANGED clause.

EXTENDS *ccfraft*

The set of COMMIT CERTIFICATES. One record per newly-committed index, frozen at the moment a LEADER advances its *commitIndex* (the only place commits originate):
index - the committed *log* index
entry - the committed *log* entry (full *ccfraft* entry record)
cterm - the COMMITTING LEADER'S term $\triangleq$ *currentTerm*[*i*] at commit time.
For an indirect (Figure-8) commit this is the HIGH term that sealed the entry, NOT the entry's own (authoring) term – exactly the distinction the entry-term form gets wrong.
cleader - the committing leader.

VARIABLE *committed*

$$fixVars \triangleq \langle vars,\ committed \rangle$$

Detect a leader commit from (state, state'): the node is a *Leader*, its *commitIndex* strictly advances, and its *log* is unchanged – this characterises *ccfraft*'s *AdvanceCommitIndex*(*i*) and excludes follower *commitIndex* updates (those happen inside *AppendEntries* handlers, which also touch *log*/messages and run on Followers). Mirrors *raft.tla* encoding #2, which records certs only at the leader's *AdvanceCommitIndex*.

$$\begin{aligned}
&LeaderCommitters \triangleq \\
&\quad \{i \in Servers : \\
&\qquad \land\ leadershipState[i] = Leader \\
&\qquad \land\ commitIndex'[i] > commitIndex[i] \\
&\qquad \land\ log'[i] = log[i]\}
\end{aligned}$$

$$\begin{aligned}
&LeaderCommitCerts \triangleq \\
&\quad \text{UNION } \{\{[index \mapsto idx, \\
&\qquad\qquad entry \mapsto log[i][idx], \\
&\qquad\qquad cterm \mapsto currentTerm[i], \\
&\qquad\qquad cleader \mapsto i] \\
&\qquad\qquad : idx \in (commitIndex[i] + 1) \,..\, commitIndex'[i]\} \\
&\qquad : i \in LeaderCommitters\}
\end{aligned}$$

$$FixInit \triangleq Init \ \land committed = \{\}$$
$$FixNext \triangleq Next \land committed' = committed \cup LeaderCommitCerts$$
$$FixSpec \triangleq FixInit \land \Box[FixNext]_{fixVars}$$

Type sanity for the certificate set.

$$CommittedTypeOK \triangleq$$
$$\forall\, c \in committed :$$
$$\land\, c.index \in Nat$$
$$\land\, c.cterm \in Nat$$
$$\land\, c.cleader \in Servers$$

---

(4) Leader Completeness, COMMIT-term-keyed (Ongaro Figure 3), certificate form. *CCF* has no *elections history variable*, *so the* "every leader of a higher term" *universe is the set of CURRENT leaders*($state[s] = Leader$ *at* $currentTerm[s]$); *this is the same universe ccfraft s* own *LeaderCompletenessInv* quantifies over.

An entry committed in term $c.cterm$ must appear, at its committed index, in the *log* of every leader whose term is strictly greater than $c.cterm$. Keying the antecedent on the COMMIT term $c.cterm$ (not the entry's own term) is the fix: an indirectly-committed entry (entry-term $< c.cterm$) no longer wrongly obligates an intervening lower-term leader.

$$LeaderCompleteness \triangleq$$
$$\forall\, s \in Servers : (leadershipState[s] = Leader) \Rightarrow$$
$$\forall\, c \in committed :$$
$$c.cterm < currentTerm[s] \Rightarrow$$
$$\land\, c.index \in \text{DOMAIN } log[s]$$
$$\land\, log[s][c.index] = c.entry$$
